Educational technical brief · Privacy on macOS

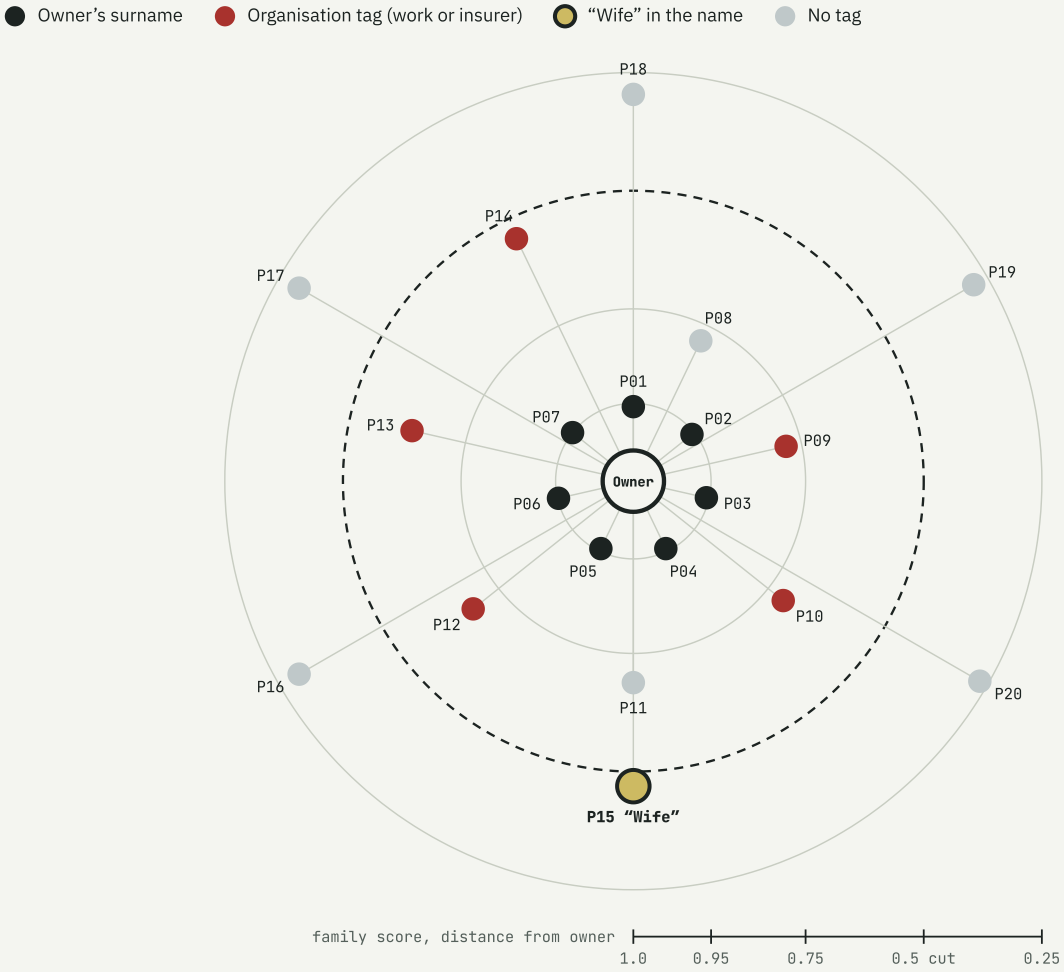
Apple Uses AI to Correctly Guess Who’s in My Family

Apple’s Personal Knowledge Graph, Verified on My Mac

Author	Date	Version	Basis
@DouglasMUN with AI assistance	10 November 2026	1.0	Pasquini & Bazyli, 8rank (28 Oct 2026), and my Mac

On 10 November 2026 I checked Dario Pasquini and Michał Bazyli’s analysis of Apple’s Personal Knowledge Graph against my Mac, using only `sqlite3` in read-only mode. Every structural claim I tested held. The store sits in `~/Library/IntelligencePlatform` behind Full Disk Access, and `entityTagging.db` scores 3,561 people across 17 relationship columns.

The scores found the family by surname and missed the spouse. All seven contacts sharing the owner’s surname scored 0.95 or higher on `family`. The contact whose name includes “Wife” scored 0.469, below seven contacts outside the family name, five of them carrying an organisation tag. Any process holding Full Disk Access can read that ranking.



How this Mac ranks the 20 people it considers possible family. The closer a person sits to the owner, the higher their `family` score; the dashed ring is iKnow’s 0.5 cut. Four work contacts and an insurer’s contact sit inside it. The spouse sits outside.

Disclaimer

This is an educational document. Its findings come from my Mac on one date, and Apple changes these schemas between macOS releases, so your stores may differ. Claims about Apple's design come from the 8rank write-up unless marked as my reading; Apple has not publicly confirmed them.

Names of real people are replaced with labels P01–P20, and entity IDs are replaced with placeholders. Run the commands only on a Mac you own, grant Full Disk Access for the session, and revoke it afterwards.

How to read the marks

Confirmed	Observed on this Mac, or read in source code.	Assessed	My reading of the data; consistent with it, not proven.
Low	Plausible, on thin evidence.	Exposure	A risk to data on your Mac.
Info	Context, or an open question.		

```
sqlite3 -readonly …
```

A command, exactly as typed in the session (shell history 7670–7678 plus the earlier commands).

```
output
```

What it printed. Names are replaced with labels; long values are rounded to three places where noted.

Key findings

KF-1 The store matches the write-up Confirmed

Fifty SQLite databases (the write-up counted 51) sit in `~/Library/IntelligencePlatform`, readable only with Full Disk Access. The kinship scorecard holds 3,561 people in 17 relationship columns. Evidence: Steps 1–2.

KF-2 The role columns hold a starting value, not a prediction Confirmed

Only 20 of 3,561 people carry role values, and all 20 start from the same split: 0.125 for mother and father, 0.25 for parent. Those two columns say nothing about anyone. Evidence: Step 3.

KF-3 Surname decided the top seven Assessed

All seven contacts sharing the owner's surname score 0.95 or higher on `family`. Four of them score exactly 0.956 and 0.879, which means the model saw identical inputs for different people. Evidence: Results.

KF-4 The spouse scored as a sibling Confirmed

The contact whose name includes "Wife" scores 0.469 on `family`, below seven contacts outside the family name. Her sibling score, 0.937, is the highest role score on the Mac, and her partner score is exactly half of it. Evidence: Step 3, Results.

KF-5 Work traffic reads as closeness Assessed

Five of the seven non-surname contacts above iKnow's 0.5 cut carry an organisation in their contact name: four work organisations and an insurer. Where surname does not apply, communication and photo features carry the score. Evidence: Results.

KF-6 Full Disk Access is the single point of failure Exposure

Accuracy does not reduce the exposure. Any process holding Full Disk Access, granted directly or inherited from a terminal, reads this ranking, and on this Mac its top 14 includes four work contacts. Evidence: For defenders.

The brief in six answers

Who	Dario Pasquini and Michał Bazyli (8rank) documented the store; I, Douglas Mun, reproduced their findings on my Mac.
What	Apple's on-device Personal Knowledge Graph: 50 SQLite stores on this Mac, including a kinship scorecard of 3,561 people.
When	Write-up published 28 October 2026; walkthrough run on 10 November 2026.
Where	`~/Library/IntelligencePlatform` on macOS. The iOS copy sits under `/private/var/mobile/Library/IntelligencePlatform`.
Why	Any process with Full Disk Access can read a ranked map of the owner's relationships, work contacts included, without crawling mail or photos itself.
How	Read-only `sqlite3` queries, joined to `graph.db` using the keys in the authors' iKnow dashboard.

Background: two daemons build a scored graph of everyone you know

The technical background is Pasquini and Bazyli's *Anatomy of Apple's Personal Knowledge Graph* (8rank, 28 October 2026), tested on macOS 26.6.2. It examines `knowledgeconstructiond` and `intelligenceplatformd`, which the authors report ship on every Mac from macOS 13. Apple documents them only in a short man page (`man 8 knowledgeconstructiond`).

Where it lives. The daemons keep 51 plaintext SQLite databases in `~/Library/IntelligencePlatform`. The only control on reading them is TCC Full Disk Access. The core store, `graph.db`, is a triple store: each row is a subject, a predicate and an object, with a confidence and its sources. Its vocabulary has 181 entity classes and 278 predicates written as opaque codes; `PS33`, for example, is a person's name copied from Contacts, and `nm_isCurrentUser` marks the device owner.

What goes in. The graph draws on Contacts, Photos face clusters, Calendar titles and attendees, Family Sharing (with each member's age), the Medical ID emergency contact, and events Siri Suggestions extracts from Mail and Messages. Separate tables hold who appears with whom in photos, a communication log of who, when and which app (no message content), location, and device signals.

How kinship is scored. Two CoreML classifiers, `EntityTagging_Family` and `EntityTagging_FamilyAndFriends`, read 144 numeric features per person. Twenty describe identity, such as a shared surname, a shared home address or iCloud Family membership; 40 describe when and how you communicate; 84 describe the photos you appear in together.

The scores go to `entityTagging.db`, one row per person with 17 relationship columns, recomputed daily. Only two of those columns have a shipped model behind them, and the authors could not trace what fills the other fifteen.

What is not forgotten. Retracted facts move to an `expired_stable_graph` table with their confidence and sources intact. The authors estimate a 48-hour retention but found much older records on their own Mac, and saw scores persist for people the graph had already dropped.

Earlier sightings. Apple's researchers described the underlying knowledge-graph techniques in a 2023 paper. A Hacker News thread asked about the same folder in 2023, Nic Lydon wrote about the database months before the 8rank post, and Alexis Brignoni's iLEAPP parses the iOS copy.

The authors also published **iKnow**, a local dashboard that reads these stores. I used its source code as a reference for the joins, without running it.

The files this walkthrough touches Info

Store	What it holds	Used in
`Artifacts/entityTagging/entityTagging.db`	The kinship scorecard: one row per person, 17 scores	Steps 2–4
`graph.db`	The triple store, including names (`PS33`)	Step 4d
`ontology.db`	Decodes predicate codes such as `PS33` into labels	Not queried

Before you start: grant Full Disk Access deliberately and read only

The terminal needs Full Disk Access to read the folder. Open System Settings → Privacy & Security → Full Disk Access, enable your terminal app, then quit and reopen it; the grant takes effect only at launch. Without it, every file in the folder returns `Operation not permitted`.

That grant is inherited Exposure. macOS attributes a process to the app that launched it, so every script, package install hook and command-line AI agent started from that terminal can read the same stores, along with Messages and Mail. Grant it for the session and revoke it afterwards.

Three rules kept the walkthrough safe:

1. **Open every store read-only.** Each `sqlite3` call uses `-readonly`, and the attached database uses a `file:` URI with `mode=ro`. The daemons hold the stores open in WAL mode and write to them while you read.
2. **Don't copy a `.db` file on its own.** Recent facts sit in the `-wal` file beside it, so a lone copy under-reports, and a stray copy outside the folder loses TCC protection.
3. **Keep the output local.** It names real people and their relationship scores. Don't paste it into a cloud-hosted agent or a ticket; iKnow's own agent manual warns that an agent sends what it reads to its model provider.

Step 1 of 4

1 Fifty databases sit in one folder Confirmed

List the folder from inside it. A successful listing also proves the terminal holds Full Disk Access.

```
cd ~/Library/IntelligencePlatform
ls
```

```
aliases.db            feedbackLog.db-wal      ontology.db
Artifacts             globalKnowledge.db      ontology.db-shm
behaviors.db          globalKnowledge.db-shm  ontology.db-wal
behaviors.db-shm      globalKnowledge.db-wal  state.db
behaviors.db-wal      graph.db                state.db-shm
eventLog.db           graph.db-shm            state.db-wal
eventLog.db-shm       graph.db-wal            views.db
eventLog.db-wal       keyvalue.db             views.db-shm
feedbackLog.db        keyvalue.db-shm         views.db-wal
feedbackLog.db-shm    keyvalue.db-wal
```

Ten databases sit at the top level, each with its `-shm` and `-wal` companions. The rest are nested under `Artifacts/`, so count recursively:

```
find ~/Library/IntelligencePlatform -name '*.db' | wc -l
```

```
      50
```

The write-up counted 51. Apple adds and renames stores between releases, so a difference of one is expected. Open The macOS version was not recorded; run `sw_vers` and note it, since schemas differ between 26.x and 27.x.

Step 2 of 4

2 The scorecard holds 3,561 people in 17 columns Confirmed

First, list the tables in the kinship store without hard-coding its path:

```
sqlite3 -readonly "$(find ~/Library/IntelligencePlatform -name entityTagging.db | head -1)" '.tables'
```

```
entity_tagging                                  has_association_subgraph_object_fts_data
has_association_subgraph                        has_association_subgraph_object_fts_docsize
has_association_subgraph_object_fts             has_association_subgraph_object_fts_idx
has_association_subgraph_object_fts_config
```

Then keep the path in a variable and read the schema, row count and a sample:

```
DB="$(find ~/Library/IntelligencePlatform -name entityTagging.db | head -1)"
echo "$DB"
sqlite3 -readonly "$DB" '.schema'
sqlite3 -readonly "$DB" 'SELECT COUNT(*) FROM entity_tagging;'
sqlite3 -readonly -header -column "$DB" 'SELECT * FROM entity_tagging LIMIT 5;'
```

The store is at `/Users/<user>/Library/IntelligencePlatform/Artifacts/entityTagging/entityTagging.db`. The schema, reformatted across lines and with the FTS5 shadow tables and triggers left out:

```
CREATE TABLE "entity_tagging" (
  "mdid" TEXT NOT NULL,
  "mother" DOUBLE, "father" DOUBLE, "parent" DOUBLE,
  "sister" DOUBLE, "brother" DOUBLE, "sibling" DOUBLE,
  "family" DOUBLE, "friend" DOUBLE, "familyAndFriends" DOUBLE,
  "partner" DOUBLE, "coworker" DOUBLE, "alumni" DOUBLE,
  "child" DOUBLE, "son" DOUBLE, "daughter" DOUBLE,
  "myself" DOUBLE, "unknown" DOUBLE,          -- all DOUBLE NOT NULL
  "eventid" INTEGER NOT NULL,
  PRIMARY KEY ("mdid") ON CONFLICT REPLACE);

CREATE TABLE "has_association_subgraph" (
  "subject" INTEGER, "predicate" TEXT, "relationshipId" INTEGER,
  "relationshipPredicate" TEXT, "object" TEXT,
  PRIMARY KEY ("subject","predicate","relationshipId","relationshipPredicate","object") ON CONFLICT REPLACE);

CREATE VIRTUAL TABLE "has_association_subgraph_object_fts" USING fts5(
  object, predicate UNINDEXED, subject UNINDEXED,
  content='has_association_subgraph');
```

What the schema shows. `entity_tagging` has one row per person keyed by `mdid`, an 18-digit graph entity ID stored as TEXT, and the 17 relationship columns the write-up describes. The other five tables are one structure: `has_association_subgraph` is a small triple table, and the `_fts` tables are an FTS5 full-text index over its `object` column. The index uses external content, which is why there is no `_content` shadow table, and three triggers keep it in sync on insert, update and delete.

The count is 3,561. The sample below has values rounded to three places and IDs replaced with placeholders; `faf` is `familyAndFriends`.

mdid	mother	father	parent	sister	brother	sibling	family	friend	faf	partner	coworker	alumni	child	son	daughter	myself	unknown	eventid
id-1	0.125	0.125	0.25	0.125	0.125	0.25	0.709	0.96	0.931	0.25	0.0	0.0	0.25	0.125	0.125	0.0	0.0	0
id-2	0.0	0.0	0.0	0.0	0.0	0.0	0.433	0.010	0.657	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
id-3	0.125	0.125	0.25	0.125	0.125	0.25	0.688	0.96	0.95	0.25	0.0	0.0	0.25	0.125	0.125	0.0	0.0	0
id-4	0.0	0.0	0.0	0.0	0.0	0.0	0.072	0.519	0.900	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
id-5	0.0	0.0	0.0	0.0	0.0	0.0	0.314	0.519	0.861	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0

What the sample shows. Rows come in two kinds. Rows 1 and 3 carry identical role values: 0.125 for each gendered role and 0.25 for each role group. Rows 2, 4 and 5 have zero in every role column and only continuous `family`, `friend` and `familyAndFriends` scores.

Rows 4 and 5 share exactly the same `friend` score (0.51937984…, which is 67/129). My reading Low is that `friend` is a ratio over shared inputs rather than a per-person model output. `eventid` is 0 in every sampled row; what it refers to is unknown.

Step 3 of 4

3 The role columns hold a starting value, not a prediction Confirmed

The sample suggested that the role columns follow a fixed pattern. Grouping every row by its role values tests that across all 3,561 people:

```
sqlite3 -readonly -box "$DB" "SELECT mother, father, parent, sibling, child, partner, COUNT(*) n FROM entity_tagging GROUP BY 1,2,3,4,5,6 ORDER BY n DESC LIMIT 15;"
```

Output, with values rounded to three places:

```
mother  father  parent  sibling  child  partner     n
0.0     0.0     0.0     0.0      0.0    0.0      3541
0.125   0.125   0.25    0.25     0.25   0.25       12
0.125   0.125   0.25    0.25     0.0    0.25        2
0.125   0.125   0.25    0.25     0.298  0.25        2
0.125   0.125   0.25    0.25     0.25   0.296       1
0.125   0.125   0.25    0.536    0.25   0.268       1
0.125   0.125   0.25    0.566    0.25   0.283       1
0.125   0.125   0.25    0.937    0.0    0.469       1
```

Only 20 of the 3,561 people have any role value; the other 3,541 are zero in every role column. In all 20, mother and father are 0.125 and parent is 0.25, so each gendered column is half of its group column. The sample rows in Step 2 show the same split for sister and brother under sibling, and for son and daughter under child, though only two rows were checked.

From that shared starting point, sibling, child and partner move for some people; parent never does. Wherever sibling has moved, partner is exactly half of it: 0.536 → 0.268, 0.566 → 0.283 and 0.937 → 0.469. My reading Assessed is that in these rows partner is derived from sibling, not scored on evidence of its own.

This partly answers what the write-up could not trace. On this Mac, most role columns hold a uniform prior, and the mother and father columns say nothing about anyone.

Step 4 of 4

4 Names live in graph.db under the same ID Confirmed

The scorecard holds IDs, not names. It took four moves to connect them to people, and the first was a dead end.

4a. The subgraph in the same file only marks the owner

The obvious candidate was `has_association_subgraph`, which sits in the same file and has a full-text index on `object`:

```
sqlite3 -readonly "$DB" "SELECT COUNT(*) FROM entity_tagging e WHERE EXISTS (SELECT 1 FROM has_association_subgraph s WHERE s.subject = e.mdid);"
sqlite3 -readonly -box "$DB" "SELECT predicate, relationshipPredicate, COUNT(*) n FROM has_association_subgraph GROUP BY 1,2 ORDER BY n DESC LIMIT 20;"
sqlite3 -readonly -box "$DB" "SELECT e.mdid, round(e.family,3) fam, round(e.familyAndFriends,3) faf, s.predicate, s.object FROM entity_tagging e JOIN has_association_subgraph s ON s.subject = e.mdid ORDER BY e.family DESC LIMIT 30;"
```

```
COUNT(*)  1

predicate         relationshipPredicate  n
nm_isCurrentUser                         1

mdid       fam  faf  predicate         object
owner      0.0  0.0  nm_isCurrentUser  true
```

The subgraph holds one fact: it marks one entity as the device owner. It carries no names, but the owner's ID is useful as an anchor into `graph.db`. The join compares `mdid` (TEXT) with `subject` (INTEGER) directly; SQLite applies numeric affinity to the text side, so the comparison works.

4b. iKnow's source shows the join

Rather than guess at `graph.db`'s schema, I read how the authors' dashboard joins the two stores, without running it:

```
git clone --depth 1 https://github.com/8rank/iKnow.git /tmp/iKnow
grep -rniE -B2 -A8 'entity_?tagging|mdid' /tmp/iKnow --include='*.py'
```

The matches established four things `Confirmed`:

- iKnow keys the scorecard by `CAST(mdid AS INTEGER)`, which equals the `subject` of rows in `graph.db` (`api/social.py`).
- The live graph tables are `stable_graph` and `event_graph`, each with an `expired_` counterpart for retracted facts (`api/overview.py`).
- `PS33` is used as the name predicate (`api/overview.py`).
- `api/kinship.py` flags score columns that are copies of others (`rollup`, `twins`) and runs nine convergence checks per person, such as the Family Sharing token, the iCloud Family flag and top contact by volume.

A later read of `api/db.py` and `api/vocab.py`, outside the shell history, added three more facts. iKnow resolves names from `PS33` (full name), then `PS312` (first name) and `PS441` (family name). It opens every store in place through a `file:…?mode=ro` URI and writes nothing; the flattened `Artifacts__…` file names are its naming scheme for extracted bundles, not copies. When run, it also reads Messages (`chat.db`), Calendar, the Photos library and Contacts, not only the graph.

4c. List the people with a role prior

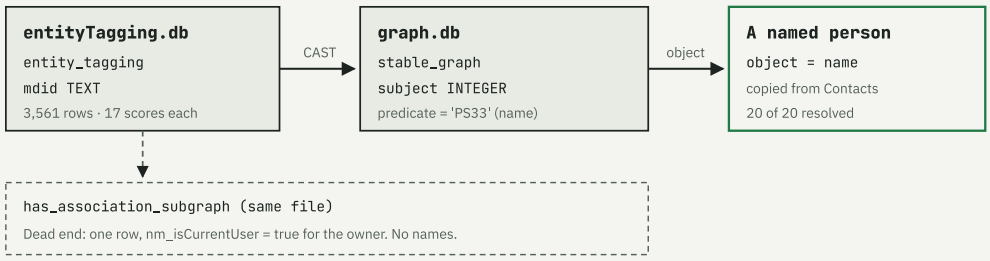
```
sqlite3 -readonly -box "$DB" "SELECT mdid, round(family,3) fam, round(familyAndFriends,3) faf, round(sibling,3) sib,
round(child,3) chi, round(partner,3) ptn FROM entity_tagging WHERE parent > 0 ORDER BY family DESC;"
```

This returned the 20 rows from Step 3 with their IDs. Their scores appear in the Results table.

4d. Attach the scorecard to graph.db and pull names

```
G=~/Library/IntelligencePlatform/graph.db
sqlite3 -readonly -box "$G" "
ATTACH 'file:$DB?mode=ro' AS t;
SELECT e.mdid, round(e.family,3) fam, round(e.familyAndFriends,3) faf,
       (SELECT group_concat(DISTINCT s.object) FROM stable_graph s
         WHERE s.subject = CAST(e.mdid AS INTEGER) AND s.predicate = 'PS33') AS name
FROM t.entity_tagging e
WHERE e.parent > 0 OR e.family >= 0.5
ORDER BY e.family DESC;"
```

The query opens `graph.db` read-only and attaches the scorecard through a read-only URI. For each scored person it looks up their `PS33` names in `stable_graph`. The `WHERE` clause adds anyone scoring 0.5 or more on `family` without a role prior. It returned the same 20 people, so nobody outside the prior group clears 0.5, and every one of the 20 had a name.



The join that worked, and the dead end beside it. The scorecard's TEXT `mdid`, cast to an integer, is the `subject` of a person's rows in `graph.db`.

Results: surname decided the top seven; the spouse scored as a sibling

The 20 people with a role prior, from Steps 4c and 4d, sorted by `family` score. The second column says what the contact name itself shows; real names are withheld. The spouse's row is shaded.

Label	Contact name shows	family	familyAndFriends	sibling	child	partner
P01	Owner's surname	0.957	0.889	0.250	0.250	0.250
P02	Owner's surname	0.956	0.879	0.250	0.250	0.250
P03	Owner's surname	0.956	0.879	0.250	0.250	0.250
P04	Owner's surname	0.956	0.879	0.250	0.250	0.250
P05	Owner's surname	0.956	0.879	0.250	0.250	0.250
P06	Owner's surname	0.952	0.956	0.250	0.000	0.250
P07	Owner's surname	0.950	0.950	0.250	0.000	0.250
P08	No tag	0.785	0.950	0.250	0.250	0.250
P09	Work-organisation tag	0.783	0.950	0.250	0.250	0.250
P10	Insurer's name as tag	0.709	0.931	0.250	0.250	0.250
P11	No tag	0.688	0.950	0.250	0.250	0.250
P12	Work-organisation tag	0.681	0.950	0.250	0.250	0.250
P13	Work-organisation tag	0.634	0.950	0.250	0.250	0.250
P14	Work-organisation tag	0.545	0.918	0.250	0.250	0.250
P15	"Wife" in the name	0.469	0.469	0.937	0.000	0.469
P16	No tag	0.298	0.298	0.250	0.298	0.250
P17	No tag	0.298	0.298	0.250	0.298	0.250
P18	No tag	0.296	0.296	0.250	0.250	0.296
P19	No tag	0.283	0.283	0.566	0.250	0.283
P20	No tag	0.268	0.268	0.536	0.250	0.268

Surname decided the top seven Assessed. Every contact sharing the owner's surname scores 0.95 or higher on `family`. Four of them score exactly 0.956 and 0.879. Identical outputs for different people mean the model saw identical inputs; my reading is that identity features such as a shared surname or iCloud Family membership outweighed behaviour for these four. P06 and P07 are the only two whose child score dropped to zero.

Work contacts outrank the spouse Confirmed. P08–P14 share no surname with the owner, yet score 0.55–0.79 on `family`, above iKnow's 0.5 cut. Five of these seven carry an organisation in their contact name: four work organisations and one insurer. Where surname does not apply, the 40 communication and 84 photo features carry the score, and my reading Assessed is that frequent work messaging is being read as closeness.

The spouse scored as a sibling Confirmed. P15's contact name includes "Wife", yet she scores 0.469 on `family`, below all 14 people above her. Her sibling score of 0.937 is the highest role score anyone received, and her partner score is exactly half of it.

The bottom six are copies, not scores Confirmed. For P15–P20, `family` equals `familyAndFriends`, and both equal one role column: child for P16 and P17, partner for P18, and half of sibling for P15, P19 and P20. Neither score is an independent model output for these people, which iKnow also flags as a rollup.

Open questions

Each gap below limits a finding. Closing it takes one more check on the same Mac.

Gap	Limits	How to close it
Is P15 declared as the owner's spouse in Contacts (related names), Medical ID or Family Sharing?	KF-4	Check the contact card and Medical ID. iKnow's documentation lists all three as model inputs; if she is declared, the classifier overrode explicit ground truth. iKnow's kinship page shows the nine per-person checks behind her row.
Which macOS version produced these stores?	KF-1, KF-2	Run `sw_vers` . iKnow is tested on 26.6.2, 26.7.1 and 27.0.1, and schemas differ between them.
What fills the fifteen columns with no shipped model?	KF-2	Repeat Step 3 on more Macs. My Mac shows a uniform prior plus a few moved values; it cannot show the rule.
Does turning off "Learn from this application" clear existing scores?	KF-6	Turn it off for Messages, Mail, Photos and Contacts, wait a day for the daily recompute, then re-run Step 3.
What does `eventid` refer to?	None	Unknown; it was 0 in every sampled row.

For defenders: Full Disk Access is the single point of failure Exposure

There is no CVE here and nothing to patch. The store works as designed and stays on the device. The risk is concentration, which the write-up calls a privacy single point of failure: any process with Full Disk Access, granted directly or inherited from a terminal, can read a pre-joined, ranked map of the owner's relationships instead of building one from raw mail and photos.

Accuracy does not reduce that risk. On this Mac, the top 14 of the ranking includes four contacts tagged with work organisations. On a security practitioner's Mac, that is a ready-made list of colleagues and where they work.

For incident responders, the same store is evidence. It records relationships, scores and retracted facts, and iLEAPP already parses the iOS copy under `/private/var/mobile/Library/IntelligencePlatform` .

There is no supported off switch. Turning off Apple Intelligence or Siri does not stop the daemon, and `launchctl disable` works only with System Integrity Protection off, which trades a privacy problem for a system-wide security one.

Action	Owner	Horizon
Audit the Full Disk Access list on managed Macs, and treat every entry as able to read this graph, Messages and Mail.	Mac fleet administrators	Next access review
Remove Full Disk Access from everyday terminals and IDEs; keep a separate terminal app for the occasions that need it.	Each Mac user	Now
Treat command-line AI agents and package installs launched from a terminal with Full Disk Access as holders of that access.	Security operations	Now
Turn off "Learn from this application" for Messages, Mail, Photos and Contacts (System Settings → Apple Intelligence & Siri → About Siri, Dictation & Privacy), then re-run Step 3 to see whether scores change.	Each Mac user	This week
Don't disable System Integrity Protection to stop the daemon.	Mac fleet administrators	Standing rule

Clean-up: revoke the access you granted

1. Remove the iKnow clone:

```
rm -rf /tmp/iKnow
```

2. Revoke the terminal's grant: System Settings → Privacy & Security → Full Disk Access, turn the terminal off, then quit and reopen it. Confirm that `ls ~/Library/IntelligencePlatform` now returns `Operation not permitted`.
3. Delete any saved copies of query output that contain names, such as terminal logs, notes and chat transcripts. The shell history holds only the commands, not their output.

Correcting the social-media version Info

A widely shared summary of the research presented 0.125, 0.25, 0.5 and 0.96 as four confidence levels, and credited Apple's daemon with a nine-signal cross-check. On this Mac, 0.125 and 0.25 are the same starting value for every candidate (Step 3). The nine-signal check is iKnow's convergence panel in `api/kinship.py`: it is how the researchers checked Apple's scores, not part of Apple's classifier.

Sources

1. Dario Pasquini and Michał Bazyli, *Anatomy of Apple's Personal Knowledge Graph*, 8rank, 28 October 2026. Technical background throughout.
   https://8rank.ai/game/apple-intelligence-personal-knowledge-graph
2. 8rank, iKnow source code: `api/social.py`, `api/overview.py`, `api/kinship.py`, `api/db.py`, `api/vocab.py` and `README.md`. Join keys, name predicates and read-only behaviour.
   https://github.com/8rank/iKnow
3. Alan West, *Turning Off Apple Intelligence Won't Stop Your Mac Guessing Who's Family*, DEV, 9 November 2026. Which settings do and do not stop the daemon.
   https://dev.to/alanwest/turning-off-apple-intelligence-wont-stop-your-mac-guessing-whos-family-adb
4. *Ask HN: Does macOS create a behavioral analytics database with your contacts?*, Hacker News, 2023. An earlier sighting of the same folder.
   https://news.ycombinator.com/item?id=37521306

Command index Info

Step	Shell history	What it does
1	Before 7669	List the folder; count databases recursively
2	Before 7669	List tables; set `$DB`; read schema, count and sample
3	7670	Group all rows by role values
4a	7671–7672	Test the subgraph as a name source
4b	7674	Clone iKnow and read its join code
4c	7675	List the 20 people with a role prior
4d	7678	Attach the scorecard to `graph.db` and pull names
Clean-up	Not yet run	Remove the clone; revoke Full Disk Access

OPENING

PLAYED BY THE BISHOP (DARIO PASQUINI) + THE KNIGHT (MICHAŁ BAZYLI) 2026-10-28

Anatomy of Apple's Personal Knowledge Graph



Every Mac since macOS 13 (allegedly) ships a pair of daemons whose only purpose in life is to build a graph about who you are, whom you know, how those people are related to you and to each other, how you communicate with them, where you go, what you do and when. In this post, we look into how this system works and give you a tool to peek into your own graph.

TARGET	AFFECTED	CLASS
Apple Intelligence / Personal Knowledge Graph	macOS	Privacy Personalized AI

What if we told you that every Mac ships a pair of daemons whose only purpose in life is to build a graph about you and the people around you: who you are, whom you know, how those people are related to you and to each other, how you communicate with them, where you go, with whom, what you do and when. This graph is built on device by cross-referencing pretty much everything on the machine: Contacts, Calendar, Photos, Messages, Mail, location, Screen Time and a long tail of behavioral signals. And, more importantly, it is held as a queryable store that LLMs retrieve from (allegedly). Would you believe us?

Well, that's what is happening, and interestingly enough, there is little documentation of how this whole mechanism works on your Mac. The closest thing to an official description is the man page (`man 8 knowledgeconstructiond`), and this is all you get:

```
MAN 8 KNOWLEDGECONSTRUCTIOND                          MACOS 26.6.2, BUILD 25G83

NAME
     knowledgeconstructiond – A daemon that analyzes content on the
     device to build a general purpose knowledge graph.

DESCRIPTION
   knowledgeconstructiond analyzes the content on this device to help
   build a knowledge graph representing the device and the user. It
     works together with intelligenceplatformd, performing some of the
     heavy processing tasks, to generate the knowledge graph.
```

Previous work: To be fair, Apple's own researchers did publish a paper on the techniques behind this graph construction, back in 2023. But there is next to nothing on how it is integrated into macOS: which versions run it, and the exact list of what is collected and inferred. Also, we personally ran into it in August 2026; but a few months before, Nic Lydon

came across the same database and wrote it up in I Built a Personal Knowledge Graph. Apple Had Already Built One on My Laptop. And even more recently, a forensics tool went looking on the phone: in September 2026, Alexis Brignoni taught iLEAPP to read the same store on iOS, where `graph.db` and the `ontology.db` that decodes its predicate codes sit under `/private/var/mobile/Library/IntelligencePlatform`; he writes it up in Reading the iPhone Knowledge Graph, and Checking Its Work. I mean, if forensics folks know about this, we don't see why you should not...

But we go a bit deeper.

This blog post is an account of some analysis work we carried out on what the knowledge graph actually does, so that you (the user) can get a better understanding of how this mechanism works, and some clarity on what the graph holds about you and your family and friends. Everything in it was collected on macOS 26 (26.6.2 at the time of writing).

To this end, we open-source a local dashboard (iKnow) you can use to read the content of your personal knowledge graph on your Mac.

Finally, we derail a bit and look into what we do best: understand how attackers might exploit it. We run a thought experiment (one that applies to any such personalized system) and show what happens if such a knowledge base ends up in the wrong hands, and how a "privacy single point of failure" can be exploited to threaten you in ways you may not have considered before (when combined with an abliterated LLM). We stress that this goes beyond the specific Apple personal knowledge graph, but it generally applies to any such personalized ML / assistant systems (Muse, Dot, etc.)

WHAT'S IN THE PERSONAL KNOWLEDGE GRAPH?

The two daemons keep their work in `~/Library/IntelligencePlatform`, a directory of SQLite databases (fifty-one databases) stored in plaintext but located in a zone protected by TCC Full Disk Access.

Likely, the most interesting is `graph.db`. It is a triple store written in a vocabulary of 181 entity classes and 278 predicates: every row is a subject, a predicate and an object. The subject is an entity (a person, an app, an event); the object is either a plain value (a name, a handle, a date) or another entity, which is where the graph gets its edges. The predicate is the interesting part: it says what kind of relationship the system believes holds between the two.

AN EDGE, AND THE ROWS IT IS STORED AS



STABLE_GRAPH · THE SAME EDGE, AS THE TWO ROWS IT IS

subject	predicate	relationshipId	relationshipPredicate	object
9100000000007919	PS500	4412	PS1	CS165
9100000000007919	PS500	4412	PS45	9100000000013724

every row also carries confidence, sources, sourceDuplicates and timestamp

One edge of the graph (synthetic data): a person and the organization the store holds as their employer. `PS500` is the id `predicate.json` gives *employer*, and its range is not an organization but a *relationship type* — so one edge is two rows sharing a `relationshipId`, one naming the kind of relationship and one pointing at the other entity. The forty-six *relationship type* classes in the table below exist to carry exactly that, which is what lets an edge hold a confidence of its own and be retracted later.

Most of what the two daemons do is populate those predicates, and the tables kept beside them, from everything the OS will let them read. Into the graph go Contacts; Photos (faces); Calendar (titles and attendees); Family Sharing (including the age of every member); the Medical ID emergency contact in Health; events and contacts Siri Suggestions finds in Mail and Messages; people donated by apps through App Intents; and installed apps. Beside it, in tables of their own, sit who appears with whom in your photos; the communication log (who, when, which app, which direction; no content), which also pulls from your other devices; location and movement; and a long tail of device signals (screen locked, Wi-Fi, Bluetooth, charging, ambient light, and a stream named `ContextualUnderstanding.PhysicalSocial` whose record format lists people by name, phone and email, each with a start and an end time; it reads like a log of who was physically around you, though we never saw it filled in).

WHAT IS INFERRED

The complete list of the 181 entity classes:

THE 181 ENTITY CLASSES IN CLASS.JSON

CLASS	LABEL	GROUP	MECHANISM FOUND?
`SB56`	entity	core	implemented
`SB104`	person	core	implemented
`SB152`	place	core	implemented
`nm_topic`	topic	core	implemented
`SB65`	event	core	implemented
`nm_document`	document	core	could not find mechanism
`SB473`	home	core	could not find mechanism
`SB10`	organization	core	implemented
`SB2`	group of humans	core	implemented
`SB144`	software	core	implemented

`SB193`	organism	core	implemented
`SB198`	agent	core	implemented
`SB83`	concept	core	implemented
`SB400`	occurrence	core	implemented
`SB60`	role	core	implemented
`SB197`	siribase type	core	implemented

Every class the shipped ontology declares, grouped rather than left in file order: the core types, then people, attributes, places and events, then the activity block, and last the forty-six *relationship types*, which are not things at all — each one reifies an edge so that the edge can carry facets of its own, a confidence among them. The ids and labels are Apple's; the grouping is ours. `intValue` runs 1 to 185 with four numbers missing — 18, 54, 169, 181 — and nothing on the disk says what they were.

Fifty-seven of the 181 describe an activity: *sleeping*, *exercising*, *mindfulness*, *going to bed*, *commute*, *dining*, most of them with a matching *type* class beside them, so nearly a third of the vocabulary went on what you are doing rather than on who or where you are. That said, it is not clear that every class is actually used; some may be placeholders that are declared and never filled. In the table, we flag as *implemented* the classes for which we could locate a mechanism that might be responsible for the class, but those are to be taken as inference work rather than ground truth.

And this is the complete list of predicates (same disclaimer applies):

THE 278 PREDICATES IN PREDICATE.JSON				
PREDICATE	WHAT IT RECORDS	DATA SOURCE	HOW IT IS OBTAINED	MECHANISM FOUND?
`PS1`	what kind of thing this is	every source	Rule	implemented
`PS10`	genre	Apple world knowledge	Apple-shipped	implemented
`PS102`	this record is the same person/ thing as that one	every source	**Embedding model**	implemented
`PS106`	duration	Calendar, Suggestions	Copied	could not find mechanism
`PS107`	has location	Calendar, location history	Copied	might be implemented (unverified)
`PS123`	part in part-whole relationsh ip	Apple world knowledge	Apple-shipped	could not find mechanism

`PS132`	has religion	Contacts	Copied	could not find mechanism
`PS133`	religion	Apple world knowledge	Apple-shipped	could not find mechanism

Every predicate the shipped ontology declares. The first column is the id as `graph.db` stores it; the last says how a value gets there: copied from a source, joined from another record, derived by a rule, shipped by Apple as world knowledge, or produced by a model (in bold). Seven we could not establish, and the table says so rather than guessing. The ids and labels are Apple's; the plain-language readings are ours.

HOW IT IS INFERRED

The methods run the whole range. Most of the vocabulary is filled by plain copying (a name from Contacts, an attendee from Calendar) and rules over the copies; the interesting rows come from models: a pair of XGBoost classifiers decides who is family from 144 mostly behavioral features, two embedding models are one of ten signals the system uses to decide when two records are the same person, twelve analyzers in Photos score partner, parent, child, coworker and friend from who appears in your pictures with you, on which trips and holidays, and Apple's on-device LLM reads your messages to put a name and a relationship on a face (if Apple Intelligence is active).

Whatever the method, every row lands in the graph with a `confidence` attached, a number between 0 and 1.

The family scores live apart. Two CoreML models ship next to the ontology, `EntityTagging_Family.mlmodelc` and `EntityTagging_FamilyAndFriends.mlmodelc`: binary XGBoost classifiers that take 144 features and return a class and a probability over it. Their scores do not go into the triple store but into a table of their own (`entityTagging.db`, one row per person, seventeen relationship columns from *family* to *mother* to *coworker*, recomputed every day), which is what the iKnow scorecard further down reads. Only two of those seventeen columns have a shipped model behind them; what fills the other fifteen, and how any of these probabilities would become a confidence in the graph, we could not trace.

The feature names are in each model's `metadata.json`, in plain text. They are the clearest statement of what this system is that we have found anywhere. Sixteen of the 144:

SIXTEEN OF THE 144 FEATURES IN METADATA.JSON

#	FEATURE	GROUP
5	`sameLastName`	identity
7	`photosPersonAge`	identity
15	`nicknameContainsEmoji`	identity
16	`sharesHomeAddress`	identity
18	`photosAgeDifferenceCategory`	identity
19	`iCloudFamilyMember`	identity

20	`personOverallPopularity`	communication
28	`personPopularityGivenSunday`	communication
46	`personPopularityGivenNight`	communication
47	`personPopularityGivenHome`	communication
57	`top1PersonInteractionMechanism`	communication
60	`photosPersonOverallPopularity`	photos
85	`photosPersonPosteriorProbabilityGivenMorning`	photos
118	`photosPersonPosteriorProbabilityGivenSaturdayNight`	photos
123	`photosPersonPopularityGivenHome`	photos
132	`photosPersonPopularityGivenWeekend`	photos

Sixteen of the 144 inputs `EntityTagging_Family.mlmodelc` reads, numbered by their place in its `metadata.json`: the six this post names, and ten that show how the rest are cut (overall, by day, by time of day, by place). `EntityTagging_FamilyAndFriends` reads the same 144 in the same order, and every one is a number. The groups are the schema's own three blocks: twenty on who the person is, forty on when and how you communicate with them, eighty-four on the photographs you both appear in. Two of the twenty, `photosPersonAge` and `photosAgeDifferenceCategory`, are read from Photos too. The names are Apple's; the group labels are ours.

The *identity* group is the tidy story: same surname, same home address, an iCloud Family flag — and it is the small half. The bulk of the model is the other two: whom you message and who appears in your photographs, conditioned on the day of the week and the time of day, and then on both at once. `photosPersonPosteriorProbabilityGivenSaturdayNight` is a shipped feature name in a shipped classifier whose job is deciding whether someone is family.

Deleting something does not delete it everywhere.

The live graph forgets. Delete a contact and nothing re-derives them; the next run replaces the table wholesale.

But the store keeps a second table, `expired_stable_graph`, beside the live one. The retracted row moves there and keeps its confidence and its sources. How long it stays there is not clear from what we could see. It seems to go after about 48 hours (the store keeps a prune watermark, and a recent snapshot of the table reached back no further than that); but on our own Mac, we could still read very old records in it.

And a conclusion can outlive the evidence for it. On our Mac, the scorecard still scored two people the graph had dropped the day before, both of them known to it only from a calendar invitation.

LOOK INTO YOUR OWN DATA

Is any of this sensitive? We'll leave that to you, since "sensitive" means different things to different people (though we think you have plenty to choose from here). However, the thing is, none of it lives in isolation. It is built to be joined and correlated; the nodes are there to be linked:

A PERSONAL KNOWLEDGE GRAPH (SYNTHETIC DATA)



me · the owner of this Mac · 1 a group the store holds as an entity · 1 family, as scored · 7
scored close to me · 9 employer · 4 group chat · 6 known, not scored close · 61
membership of a group chat a calendar event both attended an employment statement
named by the stored group a closeness score above the cut · nobody typed it

HOW TO READ THIS

Every node is a row in the store, and the record under it is what the store holds. Size is how many rows name it; shape is what kind of thing it is; the one node in accent is you, and a ring marks the four the store scores as family. A dashed line is a conclusion nobody typed.

Click a node — or reach the graph with Tab, walk it with the arrow keys and press Enter — and its record replaces these notes.

Click a node and the record under the graph is what the store holds on it (the rows, not a summary of them). The sixty-one faint ones are the part nobody expects: people the store knows and has not placed, in it because they wrote to you once, or because they were on somebody else's invitation.

The most important point, though, is that this is not only about you (the person using the Mac). The people around you are in it too: your family, your friends, "anyone" you have talked to through your device. None of them was asked, and the store links them to you and, through the events and groups they share, to each other. A relationship is intelligence one level above

the events and groups they share, to each other. A relationship is intelligence one level above the rows it was inferred from. For instance, by correlating across our own store, we (well, an LLM working for us) could easily work out the following:

- **That there is a child in the household, and roughly how old.** The family-member types and a family circle give the household. The scorecard's child and son/daughter columns give the child's place in it. Beside it sits an age, copied from Family Sharing; and dates of birth in the store can carry the system's own word *imputed*, though what the system means by it is not clear to us.
- **Which daycare the child goes to, and both of the numbers that reach it.** Copied off contact cards. Note what is *not* there: the calendar has no school-run event, so the store holds the institution and its lines but not the hour the child is dropped off. That is still enough to call a parent from a number their phone already trusts.
- **Where the family will be, on which days, between which hours.** One calendar event, a place, two stored time slots and a title that names the family (so the guest list is in the string even where the attendee rows are empty).
- **Which of the family will be there.** The scorecard names the ones scored as family, a household entity holds them together, and a Family Sharing circle says which of them are in the same family group.
- **And, in the same file, what is installed on the machine,** down to the password manager.

It is easy to see how this could be weaponized if this single point of failure for your privacy ends up in the wrong hands. We play that (hypothetical) scenario out in a second write-up. The good thing is that this knowledge is meant to stay local on your machine, protected by the Full Disk Access permission; although it can be consumed by other services (but it is not entirely clear to us how and what can be accessed).

The reality is that the amount of information in this store is really something else: every time we dig into what the daemons infer, and how, a few new lines of investigation open up. Whatever you would rather no one knew about you or your family may well be in there somewhere, and we think you have the right to know whether it is. What you get instead: no documentation, no way to inspect the store and no general way to correct an inference that is wrong.

So we give you two tools. The first is iKnow, a local dashboard for browsing your own graph without anything leaving your Mac (the next section walks through it). The second is for going deeper. The iKnow repository ships a manual for your own agent that tells it where the databases are and how to read them, so it can go looking for whatever you are after. (Please point a self-hosted model at it, not a cloud one.)

IKNOW

We designed iKnow as a web dashboard implemented in the Python standard library; no npm and no CDN.

It is meant to be run directly on your Mac, where the databases are stored. When executed, it binds `127.0.0.1`, locates the databases in your file system (granted the necessary permissions), opens them read-only, collects the info on the fly and represents it on a dashboard. Nothing is written and nothing is logged; values go from the stores, over loopback, into a browser tab; it can be run air-gapped if you want.

We have tested it on macOS 26.6.2 and 26.7.1, and, only partially, on 27.0.1.

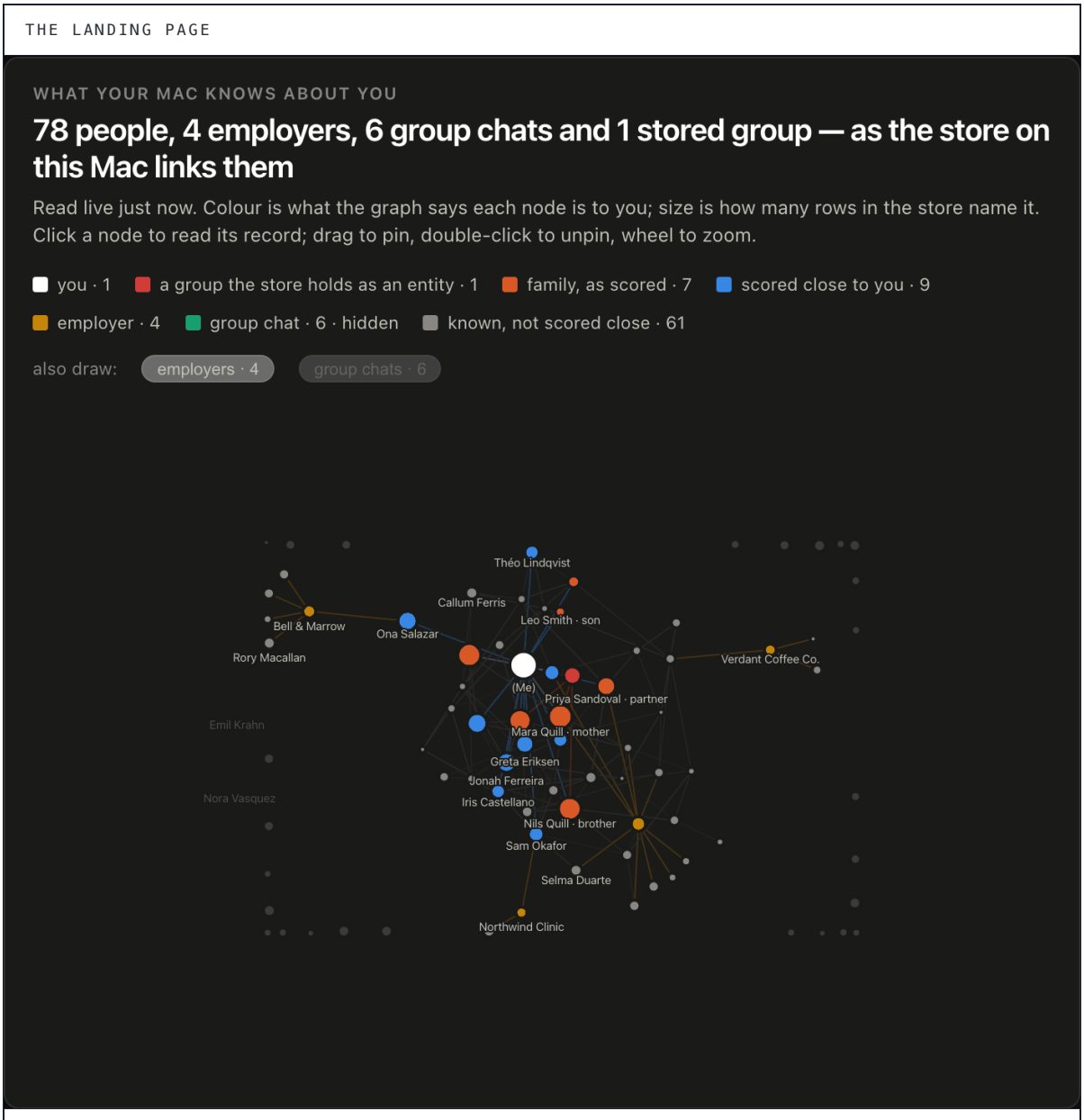
iKnow has a dozen pages.

Start here

What your Mac knows. The landing page: the graph of people around you. Below it come the faces in your photos, your family, your handles, your correspondents and where you will be next.

Its graph is the same household as the one at the top of this post, this time drawn by the tool's own renderer:

THE LANDING PAGE



The first screen. The white node marked (Me) is the owner of the Mac; the rest are the people

The first screen. The white node marked (Me) is the owner of the Mac; the rest are the people, employers and group chats the store links to it, colored by *what it says each one is*. Orange is family, as scored. Blue is a closeness score above the cut. The faded names around the edges are people the store knows and has not placed.

Nothing here is summarized that is not in the record: every node is a row, and clicking one shows you the row (which is what the graph at the top of this post does too).

The graph is the overview, and an overview is easy to shrug at. The pages behind it are not.

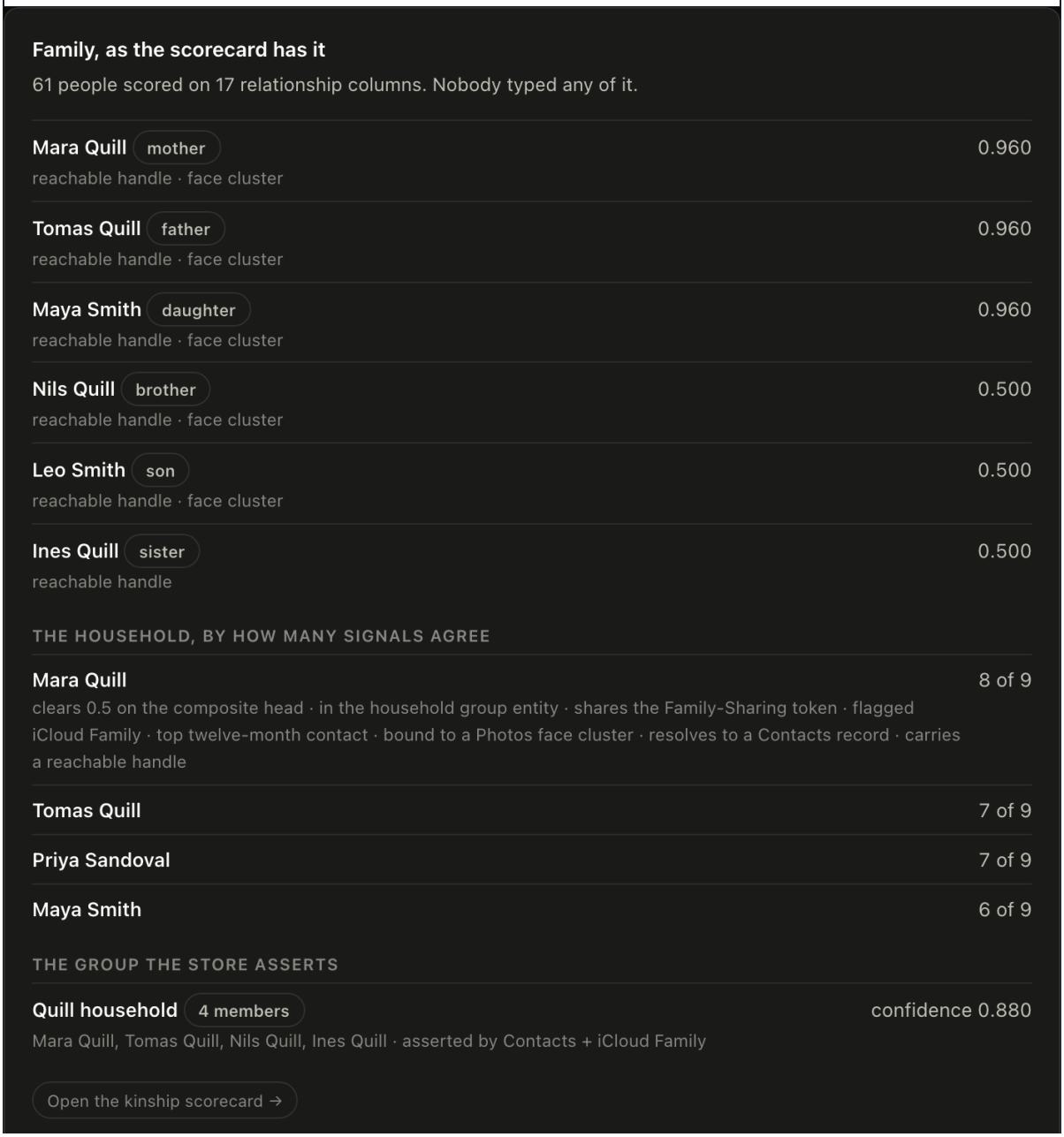
Social graph. Whom the store links to whom through shared meetings, group chats and a closeness score. Each circle opens to its members and to the evidence behind it.

What it concluded

Kinship scorecard. The family bindings inferred by the daemon.

Its card on the landing page is cut down to the people it names:

FAMILY, AS THE SCORECARD HAS IT



Six people, six relationships, and a subtitle saying nobody typed any of it. That holds for the scores; some of what they are computed from, you did type (a relation on a contact card, an emergency contact, Family Sharing). A mother, a father and a daughter at 0.960, which is as high as these columns go; a brother, a son and a sister at 0.500.

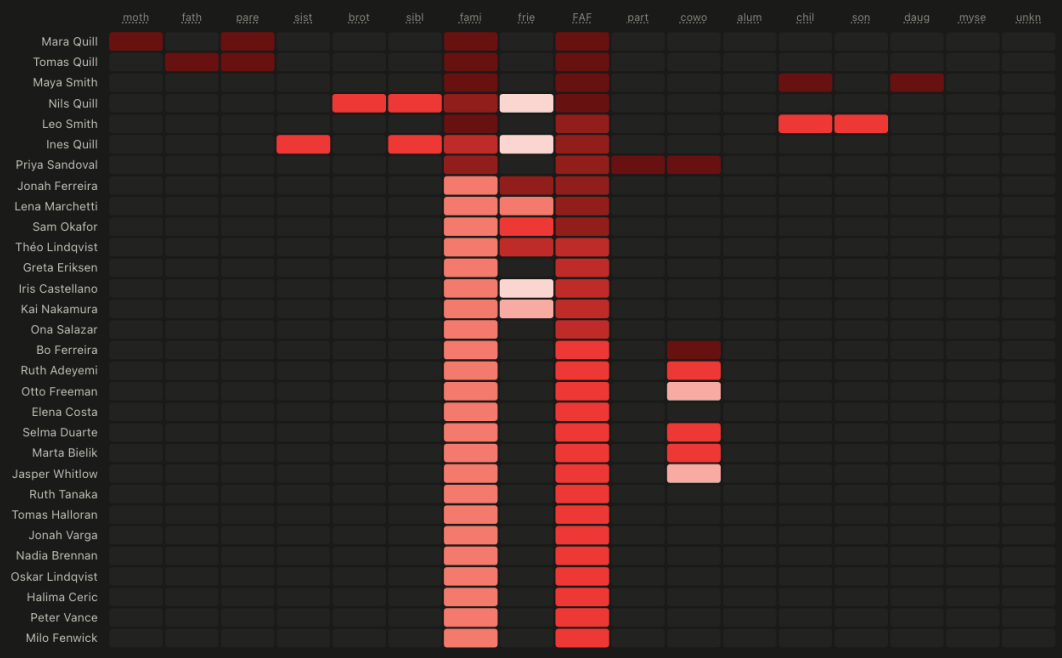
Below that is the part we find hardest to argue with. Nine separate checks (the partner column, the composite score, the household group entity, the Family Sharing token, the iCloud Family flag, top twelve-month contact, a Photos face cluster, a Contacts record, a reachable handle), and the card counts how many of them land on the same person. Eight of nine on the mother; seven on the father, and seven on Priya Sandoval, the partner, who is not in the list above: iKnow lists the relationships that roll up into another column (*mother* and *father* into *parent*), and *partner* rolls up into nothing. The join found the partner anyway.

The card is a summary of one table, and iKnow draws the whole table too:

EVERY PERSON, EVERY COLUMN

Every person, every column table

Sorted by familyAndFriends, the broadest head. Showing the top 30 of 60 ranked people — the table view lists all of them. Axis labels are abbreviated: moth = mother, fath = father, pare = parent, sist = sister, brot = brother, sibl = sibling, fami = family, frie = friend, FAF = familyAndFriends, part = partner, cowo = coworker, alum = alumni, chil = child, son = son, daug = daughter, myse = myself, unkn = unknown. 2 of the 17 columns (alumni, unknown) run flat grey from top to bottom because they are zero on every row of the table — nobody was scored low there, the columns were never used at all. 58 of 61 people resolve to a name in the graph; the rest are scored anonymously and still ranked.



Sorted by *familyAndFriends*, the broadest of the scores. The seven at the top each light up their own relationship and the column it rolls up into (*mother* and *father* into *parent*, *sister* and *brother* into *sibling*, *son* and *daughter* into *child*), and the partner is also a colleague. Further down are the friends and the rest of the office. Under them, almost everyone else is the same two cells: one repeated value on *family* and another on *familyAndFriends*, which is what the classifier writes when it has nothing to say. Two columns (*alumni* and *unknown*) are never used at all.

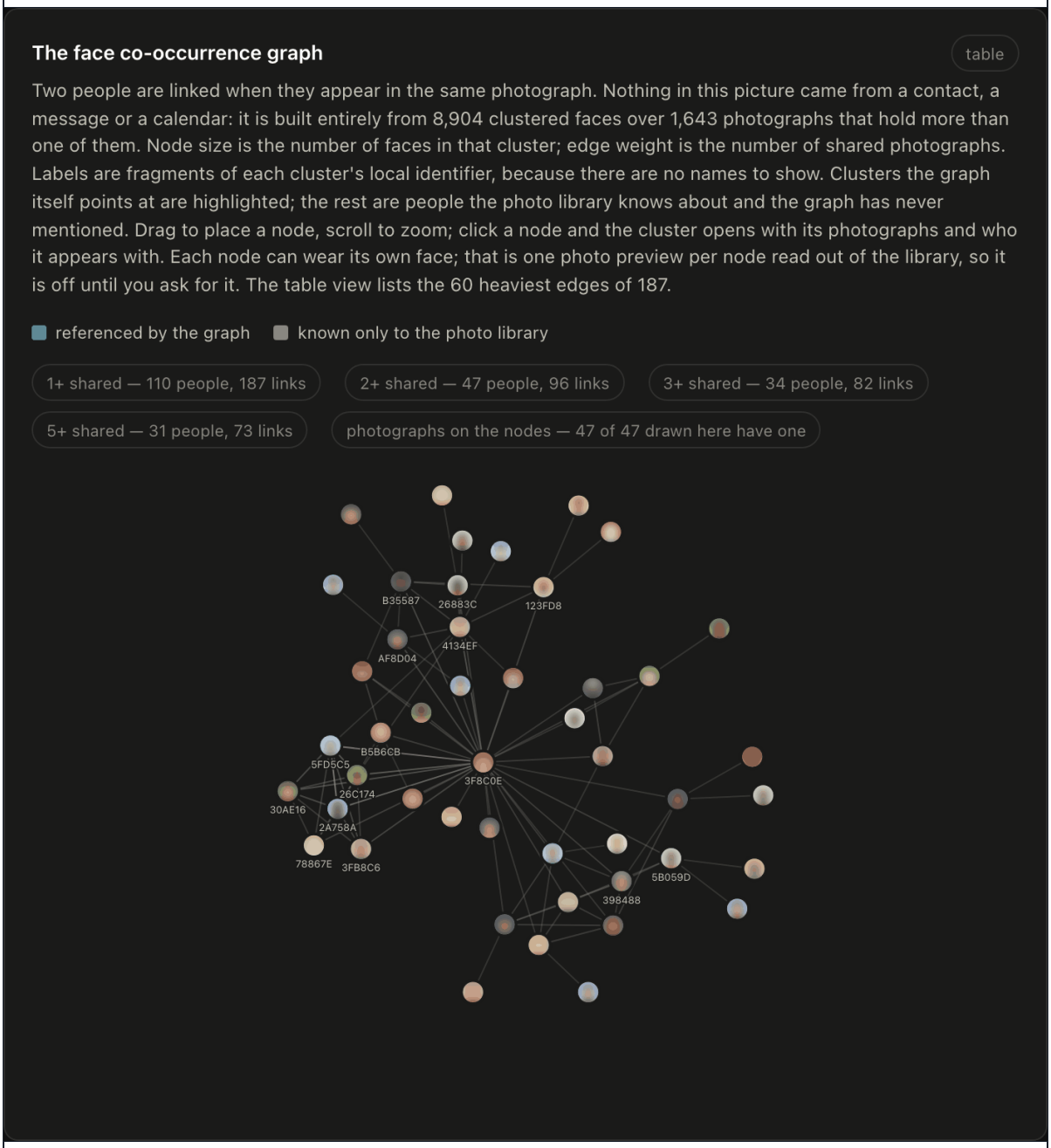
The scores in these two shots are invented, and more generous than the real thing. On the Mac we took apart, only *mother* reached the top of the ladder; the few others it scored at all

got one small vote on every family label at once, father and brother alike. What the fixture keeps is the table's grain: it only ever writes a handful of values (0.125, 0.25, 0.5 and 0.96), and most people sit on one default.

The face index. A face cluster for everyone in your photos.

Its co-occurrence graph does not use a single contact, message or calendar entry. It is built from your photographs alone:

THE FACE CO-OCCURRENCE GRAPH



Two faces are linked when they appear in the same photograph, and that is the only input: 8,904 clustered faces over the 1,643 photographs that hold more than one. The labels are fragments of each cluster's local identifier, because there are no names to show. The owner is the hub. The knot on the left is the family, the group at the bottom right is work, and the two never share a frame.

Each node wears its cluster's key face, read out of the photo library and blurred by iKnow's own switch for sharing the screen. Under the blur there is nobody: the fixture has no photo library, so every face is a synthetic portrait painted for it.

This is the page's own honest point: the graph points at the face clusters and can name almost none of them (three of the twenty-nine, in the fixture). It does not need to. Who stands next to whom, how often, and which groups never meet is all there without a single name, and whoever knows the household can put the names back on in a second.

Guessed or given. What you typed in and what the Mac worked out for itself live in the same table. A confidence score is the only thing separating them.

What it recorded

A year of contact. Whom you talked to, when, and not a word of message text (and it does not need any to tell when you are asleep).

Its card on the landing page is the one that gets the reaction, and it contains no inference at all.

A YEAR OF WHO YOU TALKED TO

A year of who you talked to

18,432 interactions over 365 days, 146 correspondents (97 named), 5 apps — and not one word of text.

Mara Quill 1,537 in · 1,363 out · 14 calls · last 2026-09-14	2,900
Tomas Quill 1,372 in · 1,218 out · 13 calls · last 2026-09-14	2,590
Priya Sandoval 1,208 in · 1,072 out · 12 calls · last 2026-09-09	2,280
Jonah Ferreira 1,044 in · 926 out · 11 calls · last 2026-09-14	1,970
Lena Marchetti 879 in · 781 out · 10 calls · last 2026-09-10	1,660
Sam Okafor 715 in · 635 out · last 2026-09-13	1,350
Théo Lindqvist 551 in · 489 out · last 2026-09-09	1,040
Ruth Adeyemi 386 in · 344 out · last 2026-09-12	730

THE HOURS IT CAN READ AS SLEEP

Quietest stretch 01:00–07:00 Europe/London: 402 of the year's 18,432 interactions, 2%, fall in those 6 hours.

Open the year of contact →

A year of contact, per person, split incoming and outgoing, with call counts and a last-seen date, across five messaging apps — and, exactly as the subtitle says, not one word of message text. None of this is content. It is all metadata, and it is enough.

The last line is the one to read twice: *the hours it can read as sleep*.

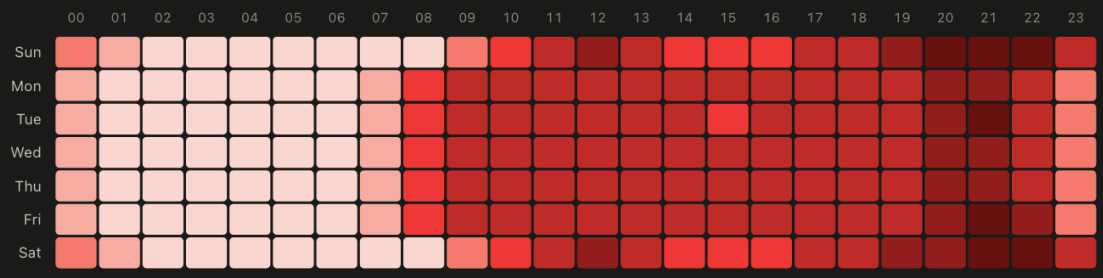
iKnow draws that line too, from the same log folded onto a single week:

iKnow draws that line too, from the same log folded onto a single week.

WEEKDAY AGAINST HOUR OF DAY

Weekday against hour of day — when this person is reachable, and when they are asleep table

18.4K message and call events, placed on a 7×24 grid; 168 of 168 cells are occupied. Hours are the host's local clock (Europe/London, UTC+01:00), resolved per timestamp so daylight saving is handled rather than smeared. Non-communication intents are excluded.



The hours the log goes quiet — a sleep schedule nobody wrote down table

Hours carrying less than 25% of an even spread form one unbroken run of 6 hours, 01:00 to 07:00 Europe/London. 402 of 18.4K events fall inside it (2%). The window is derived from the distribution, not assumed — and it is a sleep schedule, published by metadata that contains no words.

Every cell counts the messages and calls that fell in one hour of one weekday, over the year. The working week is in there, and so are the evenings and the late weekend mornings; and so is the pale band from one to seven, every night. Nobody stored a sleep schedule. The quietest six-hour stretch falls out of the timestamps for free.

That is the general shape of this store: almost nothing in it had to be collected, because almost everything in it is a consequence of something that was.

The calendar. The one layer that knows where you will be: the hour, the room, the attendees and the dial-in.

What you withdrew. Withdrawing a fact does not remove it. The row moves to an expired table and keeps its confidence and provenance.

The machinery

The store. The file itself: what it models, and two real rows expanded into everything they touch.

Device fingerprint. Every app installed on the Mac is written down in plain text, next to a token that is an unsalted SHA-256 of that same app identifier, so the page recomputes it in your browser and you can watch it match.

Who else reads it. The graph is the first copy, not the only one. This page maps the views that subscribe to it, the databases they publish into, the tables that repeat the same facts, the streams coming in from hardware paired with the Mac, and the machinery that is set up but has not run yet.

has not run yet.

A note on macOS 27. macOS 27 was released on September 14, 2026. After the upgrade to 27.0.1, the graph withdrew most of the rows it had built from Contacts, Calendar, Siri Suggestions, Family Sharing and the app catalog, and rebuilt them over the following day. Some pages of iKnow may not work fully on macOS 27 yet.

BUT LET'S GO DEEPER:

THE WHOLE GAME — 2 WRITE-UPS

- Anatomy of Apple's Personal Knowledge Graph
  - Weaponizing Personalization: From Knowledge Graph to Extortion

---

HOW TO CITE THIS GAME — BIBTEX · CLICK TO SELECT

```
@misc{8rank:apple-intelligence-personal-knowledge-graph,
  author       = {Dario Pasquini and Micha{\l} Bazyli},
  title        = {{Anatomy of Apple's Personal Knowledge Graph}},
  year         = {2026},
  month        = oct,
  howpublished = {\url{https://8rank.ai/game/apple-intelligence-personal-knowledge-graph}},
}
```